0.12 Spezielle Baumstrukturen für Zeichenketten

0.12.1 Literatur

0.12.1.1 Literatur zum folgenden Abschnitt

- Donald E. Knuth
  The Art of Computer Programming

- Robert Sedgewick
  Algorithmen in Java, Teil 1-4

- Thomas H. Cormen, Charles E. Leiserson, Ronald Rivest, Clifford Stein
  Algorithmen – Eine Einführung

0.12.2 Radix-Bäume (Tries)

0.12.2.1 Radix-Searching

- Bisher Schlüssel als Ganzes ausgewertet
- *Radix-Searching* (Digitales Suchen): Auswertung der Schlüssel zeichenweise oder bitweise
- Daten dienen selbst als Schlüssel (Gegensatz zu Transformation separater Schlüssel bei Hashing)
- Anwendungsbeispiel: lange Texte
- Voraussetzung: gut verteilte Suchschlüssel
- Vorteile
  - Effiziente Behandlung von Schlüsseln variabler Länge
  - Annehmbares Verhalten im ungünstigsten Fall

© R.-R. Grigat 2003-2019

0.12.2.2 Trie

Definition:

- Sei m die Kardinalität der Zeichenmenge C (z. B. Zahl der Buchstaben im Alphabet)
- Baum besitzt Grad $d = m + 1$
  - zusätzliches Terminierungssymbol für Ende jeder Zeichenkette benötigt
- Pfad von der Wurzel zum Blatt definiert den Schlüssel des Blattknotens, eine Kante pro Zeichen
- Name stammt von Re*trie*val

0.12.2.3 Beispiel zu Tries

Beispiel: Einfügen von HAND HANDEL WORT WO WAS

- $C = \{A, \ldots, Z\}$
- Terminierungssymbol: ∅

© R.-R. Grigat 2003-2019

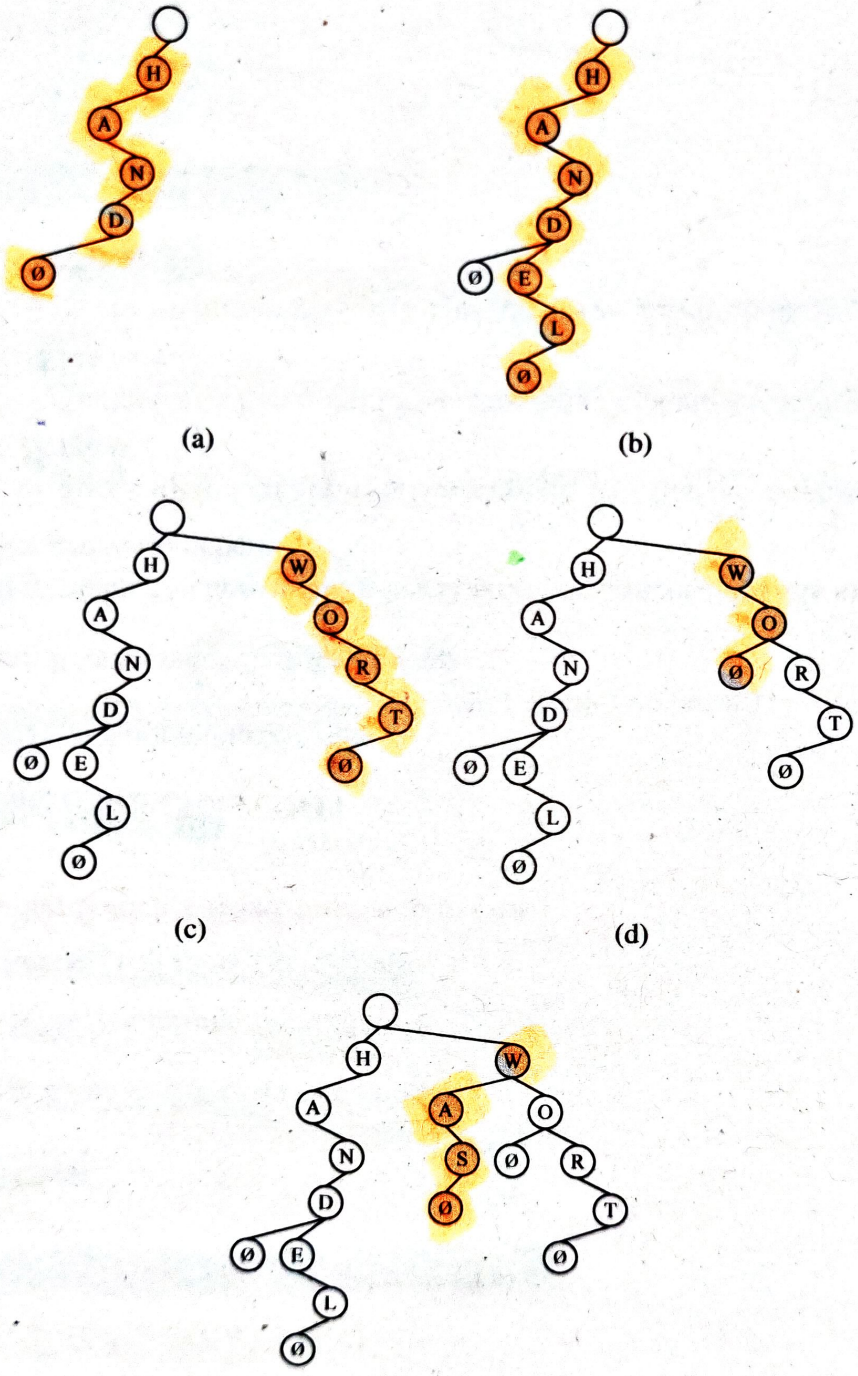


Abbildung 100: Trie

© R.-R. Grigat 2003-2019

0.12.2.4 Komplexitätsanalyse

Bisher:

- Speicherung ganzer Schlüssel (z. B. Hashing)
- Komplexität in Abhängigkeit der Anzahl der gespeicherten Schlüssel n angegeben
- ineffizient bei stark unterschiedlicher Schlüssellänge l_k

Komplexität von Tries

- Zeitkomplexität:
  - Komplexität abhängig von Schlüssellänge l_k
  - $T_{insert_{worst}}, T_{find_{worst}}(l_k) \in O(l_k)$ ähnliche Schlüssel
  - $T_{insert_{avg}}, T_{find_{avg}}(n) \in O(\log n)$ unähnliche Schlüssel
- Anzahl der Knoten im Baum:
  - Im schlimmsten Fall:
    - Alle Schlüssel disjunkt $O(n \cdot l_k)$
    - Für jeden Schlüssel l_k innere Knoten vom Grad 1
  - Durchschnittlich: $O(\frac{l_k \cdot n}{\log l_k})$ [?]

0.12.2.5 Vergleich von Trie und Hashtabelle

Trie

- Zeitkomplexität der Suche:
  - n: Anzahl der Elemente, l_k: Schlüssellänge
  - $T_{avg}(n, l_k) \in O(\log n)$ ohne Beweis, vgl. [?]
  - $T_{worst}(n, l_k) \in O(l_k)$

$\Rightarrow$ Besonders geeignet (im Vergleich zu Hashing), falls Schlüssellänge l_k *variabel*, *groß* in Relation zu $\log n$ (z. B. lange Texte) und Schlüssel gut *verteilt* sind

Hashtabelle

- Zeitkomplexität der Suche:
  - Berechnung der Hashfunktionen $h(k)$ und ggf. $h'(k)$ in $O(l_k)$, jedes der l_k Zeichen wird einzeln ausgewertet
  - $T_{\text{avg}}(n, l_k) \in O(l_k) + O(1)$
  - $T_{\text{worst}}(n, l_k) \in O(l_k) + O(n)$

$\Rightarrow$ Besonders geeignet, falls l_k *konstant* und *klein*

Nachteil von Tries

- In jedem Knoten $m+1$ Verzweigungen
  - ASCII-Codierung: $m = 256$
  - Unicode (UTF-16): $m = 65536$

$\Rightarrow$ hoher Speicherbedarf pro Knoten: $O(m)$

Abhilfe: Ternäre Such-Tries (TST)

- Jeder Knoten enthält ein Zeichen
- In jedem Knoten nur 3 Verzweigungen
  - Bei Suche Zeichen an aktueller Position des Suchschlüssels mit Zeichen im Knoten vergleichen
    - Suche im linken Sohn fortsetzen, falls Zeichen des Suchschlüssels kleiner
    - Suche im rechen Sohn fortsetzen, falls Zeichen des Suchschlüssels größer
    - Suche im mittleren Nachfolger fortsetzen, falls Zeichen des Suchschlüssels gleich

0.12.2.6 Beispiel zu ternären Such-Tries

Beispiel. Einfügen von HAND HANDEL WORT WO WAS

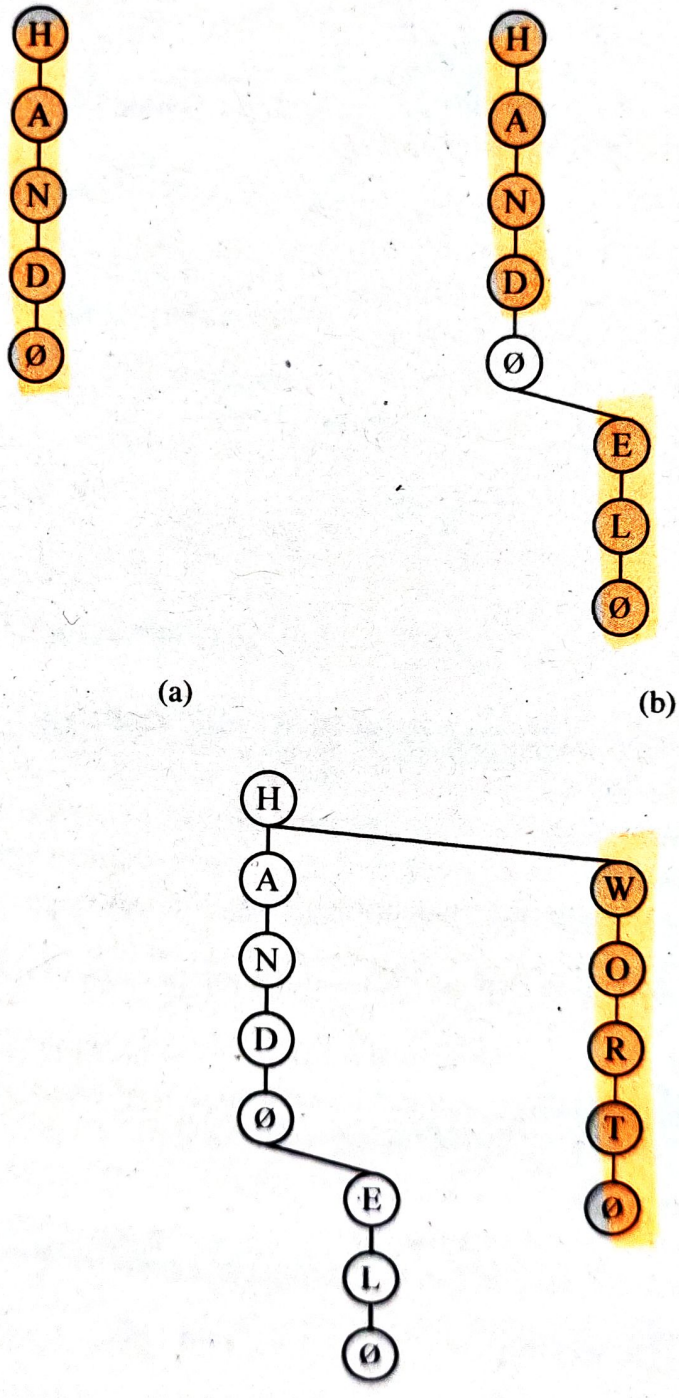
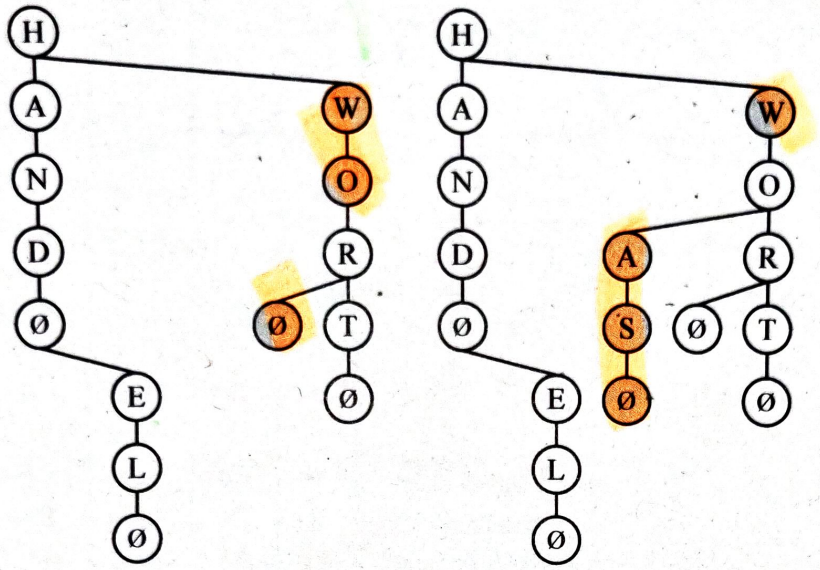


Abbildung 101: Ternärer Such-Trie

© R.-R. Grigat 2003-2019



(d) (e)

Abbildung 101: Ternärer Such-Trie (cont.)

© R.-R. Grigat 2003-2019

0.12.3 LZW Codierung

0.12.3.1 Lempel Ziv Welch (LZW) Codierung

Algorithmus von Lempel und Ziv (1978), modifiziert von Welch (1984) und Phillips (1992).

Die Codierungstabelle der LZW Codierung

- ist ein Dictionary
- ordnet Eingabe-Zeichenfolgen variabler Länge individuelle Indexwerte konstanter Bitzahl (FLC) zu
- entsteht erst während der Codierung, ist also abhängig von der Eingabe-Zeichenfolge
- wird vom Decoder aus dem Datenstrom rekonstruiert, braucht also nicht explizit übertragen werden

0.12.3.2 LZW Encoder mit Array ohne Hashing

LZW Encoder Pseudocode

```
1  Algorithmus LZW_Encoder
2  // Dictionary D leer, Zeichen x, Index d
3  for d:=0 to 255 do                  // ASCII Zeichen x zu d=0..255 eintragen
4     füge ASCII-Zeichen x zum ASCII-Index d als 1-Symbol-String zu D hinzu;
5  if (not end-of-input)
6       lies(x);                       // lies Zeichen x aus Eingabedatenstrom
7       d:= Dictionary Index von x;    // erstes Zeichen ist immer im Dictionary
8  else return;
9  endif;
10 while (not end-of-input)
11    lies(x);                        // neues Zeichen x einlesen;
12    if <D[d],x> vorhanden in D      // String um x verlängern und in D suchen
13                                    // falls verlängerter String schon vorhanden
14     d:=Dictionary Index von <D[d],x>;  // Index aktualisieren
15    else                            // falls noch nicht vorhanden
16     sende den Index d;    // Index des längsten vorhandenen Strings senden
17     füge <D[d],x> zum Dictionary hinzu;  // verlängerten String eintragen
18                           // Sonderbehandlung nötig wenn D bereits voll
19     d:=Dictionary Index von x;//last-first-property: initialisiere String
20    endif;                          //mit letztem Zeichen des vorherigen Strings
21 endwhile;
```

© R.-R. Grigat 2003-2019

0.12.3.3 LZW besitzt separate Dictionaries in Encoder und Decoder

Alphabet {i,m,p,s}, Eingabestrom m i s s i s s i p p i

Encoder		Senden	Decoder		Ausgabe
String	Index	Index	String	Index	String
i	0	-	i	0	-
m	1	-	m	1	-
p	2	-	p	2	-
s	3	-	s	3	-
mi	4	1	-	-	m
is	5	0	mi	4	i
ss	6	3	is	5	s
si	7	3	ss	6	s
iss	8	5	si	7	is
sip	9	7	iss	8	si
pp	10	2	sip	9	p
pi	11	2	pp	10	p
-	-	0	pi	11	i

© R.-R. Grigat 2003-2019

0.12.3.4 LZW Decoder mit Array ohne Hashing

LZW Decoder Pseudocode

```
1  Algorithmus LZW_Decoder
2  // Dictionary D und Strings S, Sv, out zu Beginn leer, Index d, Zeichen x
3  for d:=0 to 255 do              // ASCII Zeichen x zu d=0..255 eintragen
4     füge ASCII-Zeichen x zum ASCII-Index d als 1-Symbol-String zu D hinzu;
5  if (end-of-input) return "";      // leeren String ausgeben
6  lies (d);   // ersten Indexwert d von Eingabe einlesen
7  Sv := D[d]; // "Vorherigen" String Sv auf erstes Zeichen setzen...
8  out := Sv;  // ...und an Ausgabestring anfügen
9  while (not end-of-input)
10   lies(d);      //nächsten Indexwert d einlesen
11   if (d vorhanden in D)  // Index d in Dictionary D vorhanden?
12     S := D[d];           // "neuen" String S aus Dictionary D auslesen
13   else                   // falls d noch nicht in D vorhanden
14     S := Sv + erstesZeichen(Sv); // String S aus vorherigem String...
15   endif;                 // ...plus dessen erstem Zeichen kombinieren
16   out := out + S; // "neuen" String an Ausgabe anhängen
17   Sv + erstesZeichen(S)  an nächste freie Position in D speichern;
18   Sv := S;  // Decodierten String für nächsten Schritt sichern
19 endwhile;
20 return out; // Ausgabestring übergeben
```

Der Decoder benötigt als Initialisierung nur das Alphabet in vorgegebener Reihenfolge, also im Beispiel die ersten beiden Zeilen. Der Decoder erstellt danach die Tabelle anhand des empfangenen Datenstroms. Für jedes empfangene Symbol und zugehörigen decodierten String wird ein neuer Tabelleneintrag angelegt aus neuem String und neuem Index, wobei der neue String um ein Zeichen länger ist als der gerade vorher decodierte String. Das letzte Zeichen ist noch unbekannt. Der Decoder wartet nun auf den nächsten empfangenen Index, decodiert ihn, nimmt dessen erstes Zeichen, fügt es hinten an den vorherigen decodierten String an (Last-First-Property) und trägt das Resultat als neuen Eintrag in den Dictionary ein. Es wird also aus der letzten Spalte (Ausgabe) das erste Zeichen an die letzte Position des Strings in der Zeile darüber kopiert.

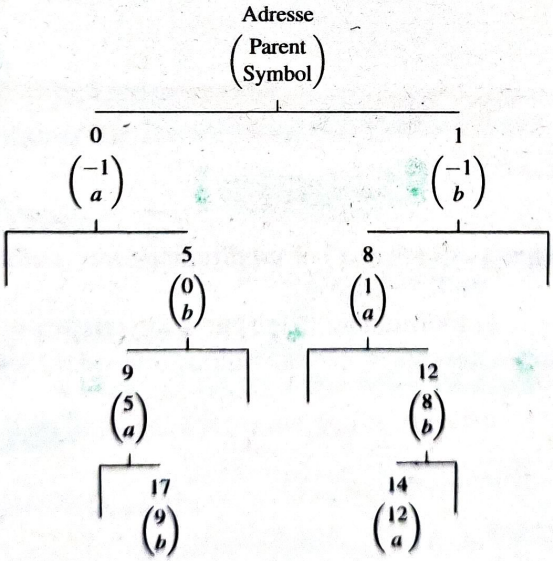
Der Sonderfall tritt beim Senden von Index 4 ein. Hier ist der Tabelleneintrag zum Index 4 erst im vorherigen Schritt entstanden und somit noch im letzten Zeichen unvollständig. Insofern ist aber das erste Zeichen zu Index 4 schon bekannt, kann an die letzte Stelle des vorherigen Ausgabestrings eingefügt werden und das Verfahren läuft wie oben beschrieben weiter.

© R.-R. Grigat 2003-2019

0.12.3.5 LZW Encoder mit Trie

Str	Hash	Stat	Adr	Par	Sym	Send
a			0		a	
b			1		b	
ab	$h(0,1)=5$	neu	5	0	b	0 (a)
ba	$h(1,0)=8$	neu	8	1	a	1 (b)
ab	$h(0,1)=5$	vorh.		0		
aba	$h(5,0)=8$	Koll.		5		
	$h_1(5,0)=9$	neu	9	5	a	5 (ab)
ab	$h(0,1)=5$	vorh.		0		
aba	$h(5,0)=8$	Koll.		5		
	$h_1(5,0)=9$	vorh.		5		
abab	$h(9,1)=17$	neu	17	9	b	9 (aba)
bab	$h(8,1)=12$	neu	12	8	b	8 (ba)
baba	$h(12,0)=14$	neu	14	12	a	12 (bab)

0.12.3.6 LZW Trie von abababababab



© R.-R. Grigat 2003-2019

0.12.3.7 LZW Decoder mit Trie

In	Str	Hash	Adr	Par	Sym	Out
	a		0	-	a	
	b		1	-	b	
0						a (0)
1	ab	$h(0,1)=5$	5	0	b	b (1)
5	ba	$h(1,0)=8$	8	1	a	ab (5)
9	aba	$h(5,0)=8$	9	5	a	aba (9)
8	abab	$h(9,1)=17$	17	9	b	ba (8)
12	bab	$h(8,1)=12$	12	8	b	bab (12)

0.12.3.8 LZW Anmerkungen

- Wenn Dictionary gefüllt ist, dann können neue Einträge nur noch alte Einträge verdrängen.
- Wenn der Decoder eine Adresse nicht im Dictionary findet, dann
  - wurde der Eintrag unmittelbar davor im Dictionary des Encoders erstellt und
  - es fehlt dem Decoder nur noch das letzte Zeichen des Strings. Dies ist aber das erste Zeichen des nächsten zu decodierenden (hier also desselben) Strings. Es sind also erstes und letztes Zeichen dieses Strings identisch.
  - Dies ist im Beispiel bei Adresse 9 und 12 der Fall.
- Jeder Knoten enthält Parent-Adresse und Index des letzten Zeichens. Hieraus kann jederzeit der Hashwert errechnet werden.

0.12.4 Huffman-Codierung

0.12.4.1 Einleitung

Gesucht:

- Effiziente binäre Repräsentation von Ereignisfolgen, z. B.:
  - Buchstaben eines Textes,
  - Pixelwerte eines Bildes

Alternativen der Codierung:

- Für jedes Ereignis separates Codewort verwenden

© R.-R. Grigat 2003-2019

  - festgelegte Codewortlänge pro Ereignis (fixed length coding FLC)
  - variable Codewortlänge pro Ereignis (variable length coding VLC)
- Verbundcodierung: Mehrere Ereignisse in einem Codewort encodieren (nicht Teil dieser Vorlesung)

0.12.4.2 Codes festgelegter Länge

Codes festgelegter Länge

- Jedem Ereignis $c \in C$ der Ereignismenge C (*Alphabet*) wird eindeutige binäre Zeichenkette (*Codewort*) der Länge $l(c)$ zugeordnet
- Für Eingabemenge C der Mächtigkeit $|C|$ beträgt die Codewortlänge:

$$l \geq \lceil \log_2 |C| \rceil$$

Beispiel: ASCII-Code

- American Standard Code for Information Interchange)
- Eindeutige Repräsentation für Buchstaben, Ziffern und einige Sonderzeichen
- Länge eines Codewortes: 8 Bit
- 01001000 01000001 01001100 01001100 01001111 =HALLO

0.12.4.3 Codes variabler Länge

Codes variabler Länge

- Codewortlänge $l(c)$ für Ereignis c ist abhängig von dessen Wahrscheinlichkeit bzw. relativer Häufigkeit $P(c)$
  - Mittlere Codewortlänge $\bar{l}$ für Alphabet C

$$\bar{l} = \sum_{c \in C} P(c) l(c) \tag{22}$$

© R.-R. Grigat 2003-2019

Beispiel: Codes für Bild mit fünf Farbarten

	rel. Häufigkeit	Code var. Länge	Code fester Länge
weiß	0,4	0	000
schwarz	0,35	11	001
rot	0,15	101	010
gelb	0,05	1000	011
blau	0,05	1001	100

Tabelle 25: Beispiel zu Codes variabler Länge

- Mittlere Codewortlänge bei *variabler* Codelänge:

$$\bar{l}_v = 0,4 \cdot 1 + 0,35 \cdot 2 + 0,15 \cdot 3 + 0,05 \cdot 4 + 0,05 \cdot 4 = 1,95 \text{ Bit}$$

- Mittlere Codewortlänge bei *fester* Codelänge ist schlechter:

$$\bar{l}_f = \lceil \log_2 5 \rceil = 3 \text{ Bit} > \bar{l}_v$$

0.12.4.4 Präfix-Codes

Definition

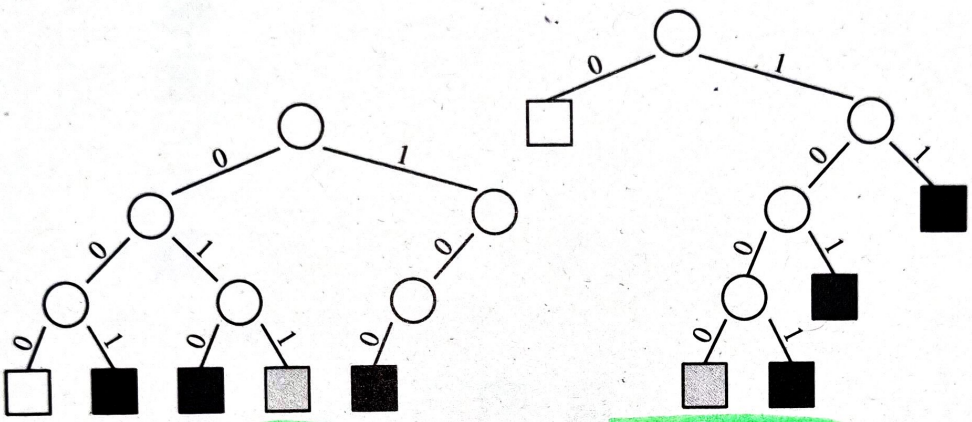
- Kein Codewort ist Präfix eines anderen Codeworts
- auch bei variabler Codelänge (VLC) kein Terminierungssymbol pro Codewort erforderlich
- Decodierung:
  - Decodierung für jedes Codewort nach Empfang seines letzten Bits eindeutig möglich
  - Bei Übertragungsfehler eines Bits ist bei VLC weitere Decodierung erst nach Resynchronisierung möglich

Beispiel zu Code variabler Länge (Tab. 25)
1001011 =*blau weiß schwarz*

© R.-R. Grigat 2003-2019

0.12.4.5 Binärer Baum für Darstellung von Präfix-Codes

Beispiel



(a) Code fester Länge (b) Code variabler Länge

Abbildung 102: Baumdarstellung der Präfix-Codes aus Tab. 25

0.12.4.6 Binärer Baum für Darstellung von Präfix-Codes

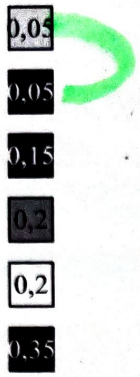
Binärer Baum für Präfix-Codes

- Elemente c des Alphabets C in *Blättern* speichern
- Schlüsselwerte der Blätter sind Wahrscheinlichkeiten $P(c)$
- Pfad Wurzel→Blatt ergibt Codewort, ein Bit pro Kante
  - Ziffer 0 in Codewort: Kante führt zu linkem Sohn
  - Ziffer 1 in Codewort: Kante führt zu rechtem Sohn
  - Pfadlänge zu Blatt c ist Codewortlänge $l(c)$
- Jede Pfadlänge (bzw. Codewortlänge) $l(c)$ von Wurzel zu Blatt wird gewichtet mit Wahrscheinlichkeit $P(c)$ des Elementes c zu $P(c)l(c)$
- Optimierungsaufgabe: Finde Baum mit *minimaler mittlerer gewichteter Pfadlänge*, vgl. Gleichung (22)

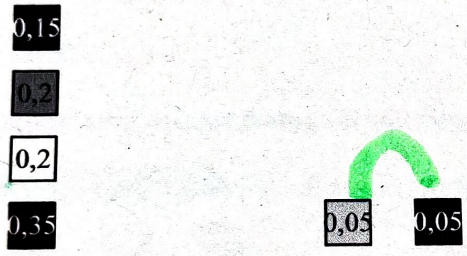
© R.-R. Grigat 2003-2019

$$\bar{l} = \sum_{c \in C} P(c) l(c) \stackrel{!}{=} \min$$

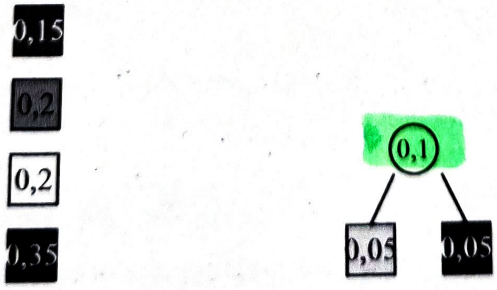
© R.-R. Grigat 2003-2019



(a)



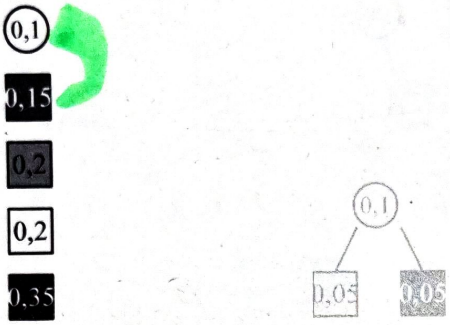
(b)



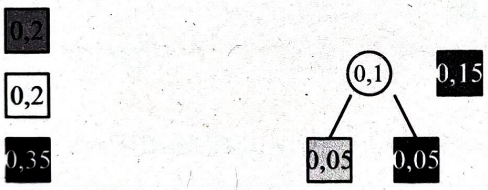
(c)

© R.-R. Grigat 2003-2019

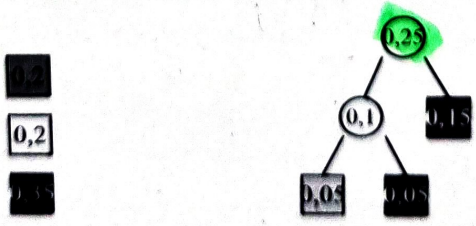
Abbildung 103: Konstruktion eines Huffman-Codes für die Farben eines Bildes mit gegebenen relativen Häufigkeiten



(d)



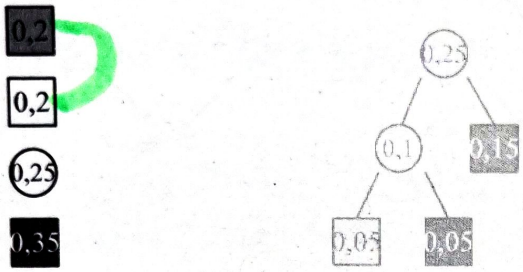
(e)



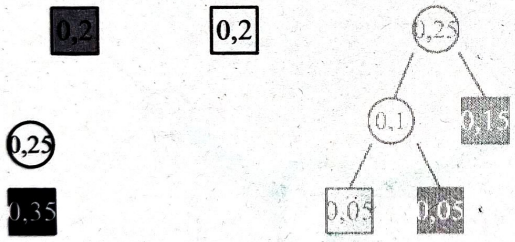
(f)

© R.-R. Grigat 2003-2019

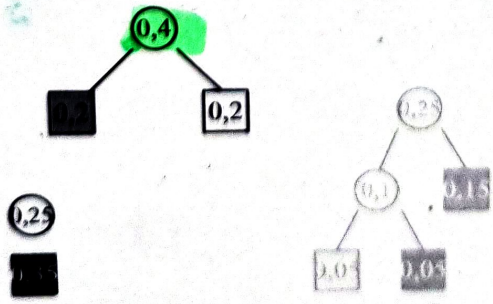
Abbildung 103: Konstruktion eines Huffman-Baums (cont.)



(g)



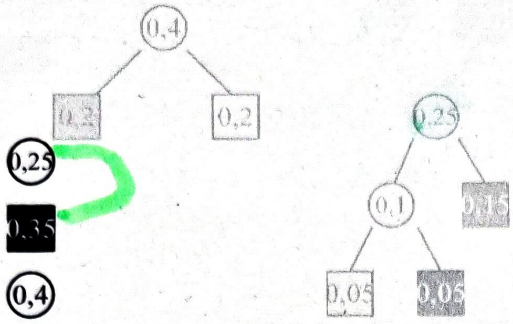
(h)



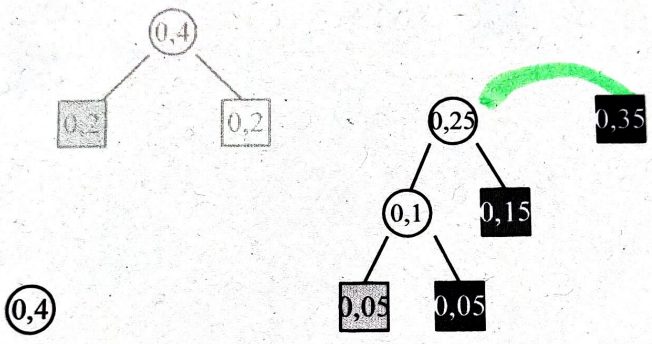
(i)

© R.-R. Grigat 2003-2019

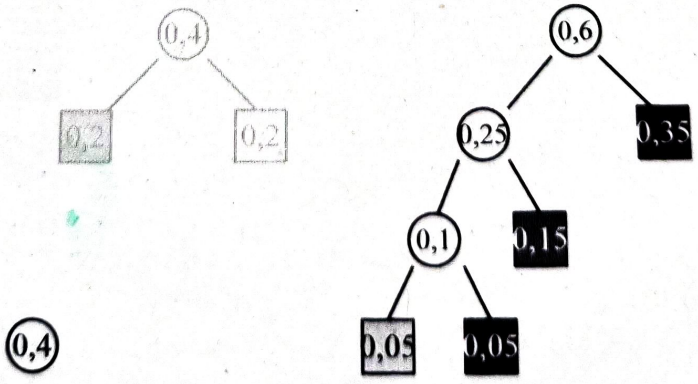
Abbildung 103: Konstruktion eines Huffman-Baums (cont.)



(j)



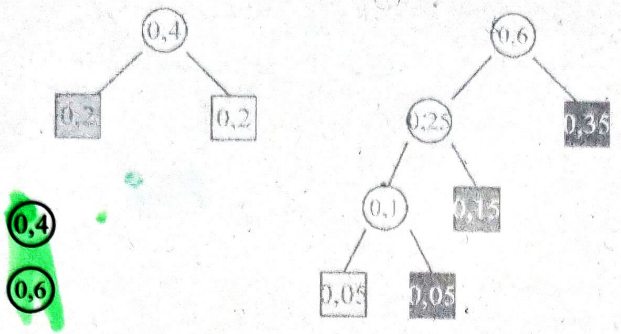
(k)



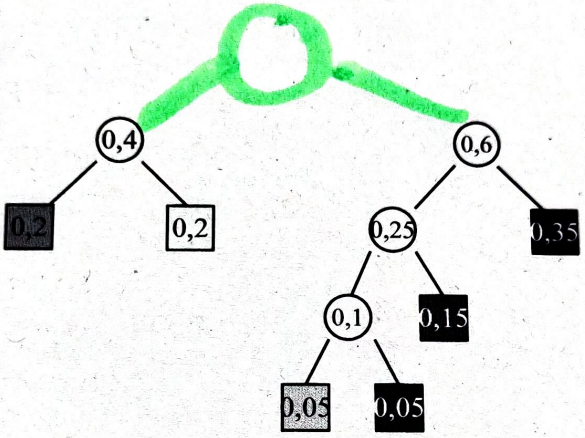
(l)

© R.-R. Grigat 2003-2019

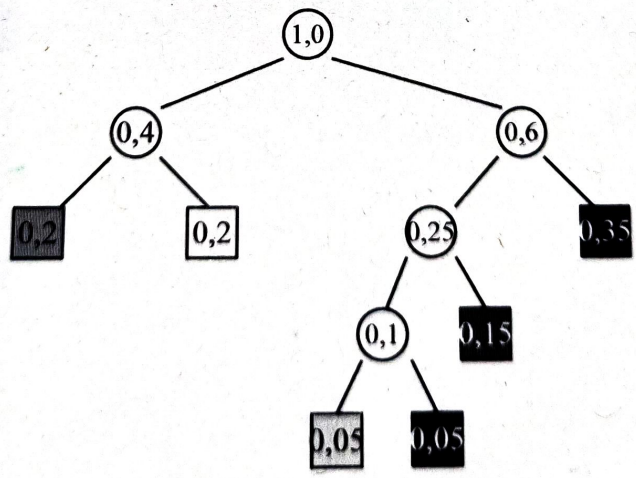
Abbildung 103: Konstruktion eines Huffman-Baums (cont.)



(m)



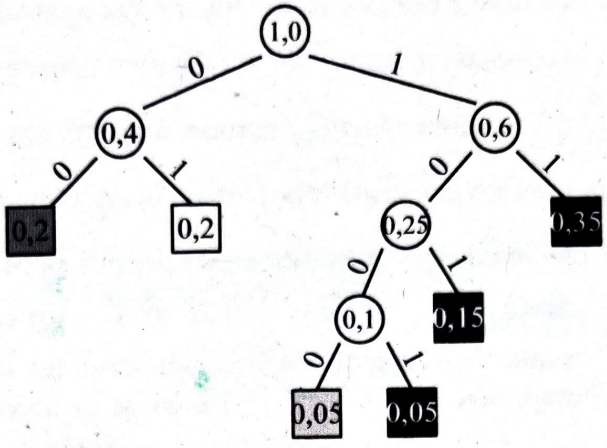
(n)



(o)

© R.-R. Grigat 2003-2019

Abbildung 103: Konstruktion eines Huffman-Baums (cont.)



(p)

Abbildung 103: Konstruktion eines Huffman-Baums (cont.)

0.12.4.7 Minimierung der mittleren gewichteten Pfadlänge

Algorithmus von Huffman

- Die Ereignisse c aus dem Alphabet C der Mächtigkeit n besitzen die Wahrscheinlichkeiten $P(c)$
- *Ziel*: Erzeugung eines Binärbaums mit Blättern $c \in C$ und *minimaler mittlerer gewichteter Pfadlänge* $\bar{l}$

```
1  Huffman (Alphabet A)   // Erzeuge Huffman-Baum aus nicht-leerer Menge A
2  { if A enthält mindestens zwei Elemente
3    then
4    { entnimm aus Menge A diejenigen zwei Elemente i, j
5              mit kleinsten Wahrscheinlichkeiten P(i) und P(j);
6      füge neues Element k mit P(k) = P(i) + P(j) zu Menge A hinzu;
7      T = Huffman(A); //Rekursive Lösung des verkleinerten Problems
8      füge in Baum T Blätter p_i mit Wert i und p_j mit Wert j
9                                    als Söhne von Blatt p_k ein;
10   }
11   else { erzeuge Baum T mit einzigem Element von A in Wurzel}
12   return T    // Liefere Baum als Resultat
13 }
```

© R.-R. Grigat 2003-2019

0.12.4.8 Komplexitätsanalyse

Komplexität T der Erzeugung eines Huffman-Baums mit n Elementen

- Sortierung der Wahrscheinlichkeiten erfolge in einem Heap
- Einfügen aller Elemente in Heap: $T_1 \in O(n \log n)$
- In jedem Schritt
  - Entnahme des minimalen Elements: $T_{deletemin} \in O(\log n)$
  - Verschmelzen zweier Teilbäume: $T_{Verschmelzen} \in O(1)$
  - Einfügen des neuen Elements in Heap: $T_{insert} \in O(\log n)$

$$\begin{aligned} T &= T_1 + (n-1) \cdot (2 \cdot T_{deletemin} + T_{Verschmelzen} + T_{insert}) \\ &\in O(n \log n) \end{aligned}$$

© R.-R. Grigat 2003-2019

0.12.5 Patricia-Bäume

Definition

- Patricia: **P**ractical **a**lgorithm **t**o **r**etrieve **i**nformation **c**oded **i**n **a**lphanumeric
  - geeignet für alphanumerisch codierte Information
  - z. B. ASCII-codierte Zeichenketten
- Patricia-Bäume sind Binärbäume
  - Innere Knoten repräsentieren Vergleiche je eines Bits
  - Blätter repräsentieren die Strings (Daten)
  - Keine Suchbaumstruktur, denn innere Knoten enthalten keine Daten
- Bei Suche werden *nur* die zur *Unterscheidung* der gespeicherten Strings unverzichtbaren Bits verglichen.
- Zahl der Blätter und inneren Knoten ist identisch (pro neuem String ein neuer Bitvergleich)
- Paare aus einem inneren Knoten und einem Blatt können gemeinsam gespeichert werden.

Bitvergleich bei Suche nach String

- Die Suche nach einem String erfolgt entlang eines Pfades von der Wurzel zum Blatt.
- Jeder innere Knoten speichert Position des zu vergleichenden Bits relativ zur Position des im Elternknoten verglichenen Bits.
  - Positionswert in Knoten gibt also an, um wie viele Bits im Suchschlüssel gegenüber Elternknoten weitergezählt werden muss.
  - Gesuchtes Bit 0, dann suche im linken Sohn weiter.
  - Gesuchtes Bit 1, dann suche im rechten Sohn weiter.
- Jedes Blatt speichert einen String (oder einen Zeiger auf einen String).
- Die Kanten zu den Blättern werden *Fäden* genannt.
  - Das Blatt enthält entweder die gesuchte Zeichenkette
  - oder die gesuchte Zeichenkette ist nicht in Baum enthalten.

© R.-R. Grigat 2003-2019

0.12.5.1 Suchen in Patricia-Bäumen

Beispiel: Suche von „WO“

- WO = 01010111 01001111 00000000
- Wortendezeichen: Nullbyte = $(00000000)_2$
- Start im Wurzelknoten
- Bit 0 links von Bits 1-24 stets 0, gehe zu *linkem* Sohn
- Weiterzählen um *4* Bits, 0+4=4
- Bit 4 = *1* ⇒ rechter Sohn
- Weiterzählen um *9* Bits, 4+9=13
- Bit 13 = *1* ⇒ rechter Sohn
- Weiterzählen um *5* Bits, 13+5=18
- Bit 18 = *0* ⇒ linker Sohn
- Kante ist *Faden* ⇒ Suche beendet
- Wenn Knoteninhalt ≠ „WO“, dann ist „WO“ nicht im Baum

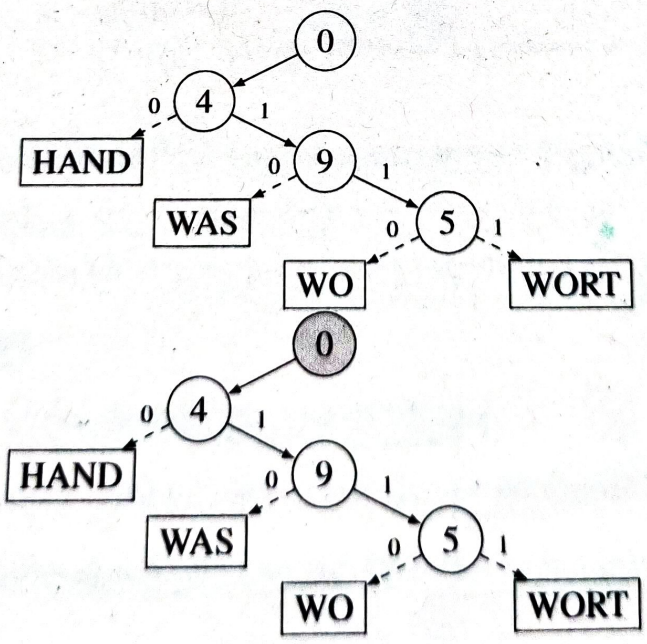


Abbildung 104: Suche in Patricia-Baum

© R.-R. Grigat 2003-2019

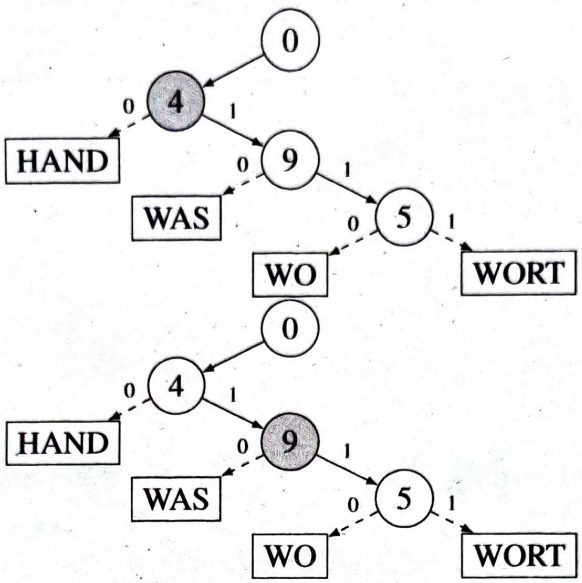


Abbildung 104: Suche in Patricia-Baum (cont.)

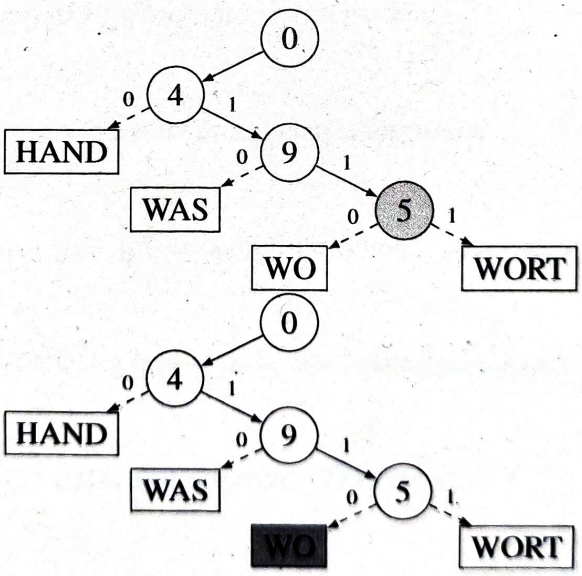


Abbildung 104: Suche in Patricia-Baum (cont.)

© R.-R. Grigat 2003-2019

0.12.5.2 Einfügen in Patricia-Bäume

Beispiel: Einfügen von HAND WO WORT WAS

- Suchen von WO im Baum,
- Linker Nachfolger: Faden zu sich selbst (Abb. 105(a))
- Wurzelknoten: Immer im linken Nachfolgerknoten suchen (Abb. 105(b))
- Nachfolger ist Faden zu HAND, WO daher als linken Nachfolger einfügen und mit HAND vergleichen
- HAND = 01001000 01000001 010011100 01000100 00000000 (Abb. 105(c))
- WO = 01010111 01001111 00000000
- Erster Unterschied in Bit $0+4 \Rightarrow$ Positionswert 4
- HAND: Bit $4 = 0 \Rightarrow$ linker Nachfolgerknoten (Abb. 105(d))
- WO: Bit $4 = 1 \Rightarrow$ rechter Nachfolgerknoten
- Wurzelknoten: Immer im linken Nachfolgerknoten suchen (Abb. 105(e))
- WORT = 01010111 01001111 01010010 01010100 00000000 (Abb. 105(f))
- Bit $4 = 1 \Rightarrow$ rechter Nachfolgerknoten
- Nachfolger ist Faden zu WO, WORT daher mit WO vergleichen
- WO = 01010111 01001111 00000000 (Abb. 105(g))
- WORT = 01010111 01001111 01010010 01010100 00000000
- Nächster Unterschied in Bit $18 = 4 + 14 \Rightarrow$ 14 Bits weiterzählen
- WO: Bit $4 + 14 = 0 \Rightarrow$ linker Nachfolgerknoten (Abb. 105(h))
- WORT: Bit $4 + 14 = 1 \Rightarrow$ rechter Nachfolgerknoten
- Wurzelknoten: Immer im linken Nachfolgerknoten suchen (Abb. 105(i))
- WAS = 01010111 01000001 01010011 00000000 (Abb. 105(j))
- Bit $4 = 1$, Bit $18 = 1 \Rightarrow$Faden zu WORT, also WAS mit WORT vergleichen
- WORT = 01010111 01001111 01010010 01010100 00000000 (Abb. 105(k))

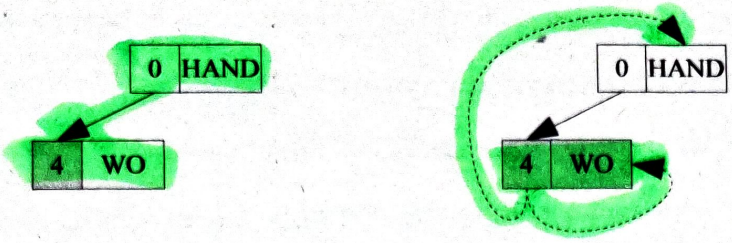
© R.-R. Grigat 2003-2019

- WAS = 01010111 01000001 01010011 00000000
- Unterschied in Bit $13 = 4 + 9 \Rightarrow$ 9 Bits weiterzählen
- WORT = 01010111 01001111 01010010 01010100 00000000 (Abb. 105(l))
- WAS = 01010111 01000001 01010011 00000000
- Unterschied in Bit $4 + 9 < 4 + 14 \Rightarrow$ als Vorgänger einfügen
- WAS: Bit $4 + 9 = 0 \Rightarrow$ linker Nachfolgerknoten (Abb. 105(m))
- WORT: Bit $14 + 9 = 1 \Rightarrow$ rechter Nachfolgerknoten
- Positionswert in Nachfolgerknoten aktualisieren

(a) (b)

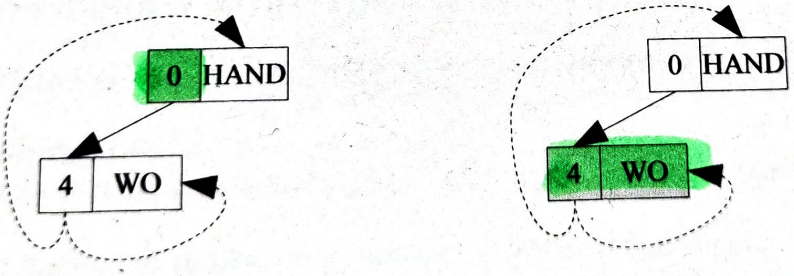
Abbildung 105: Einfügen in Patricia-Baum

© R.-R. Grigat 2003-2019



(c) (d)

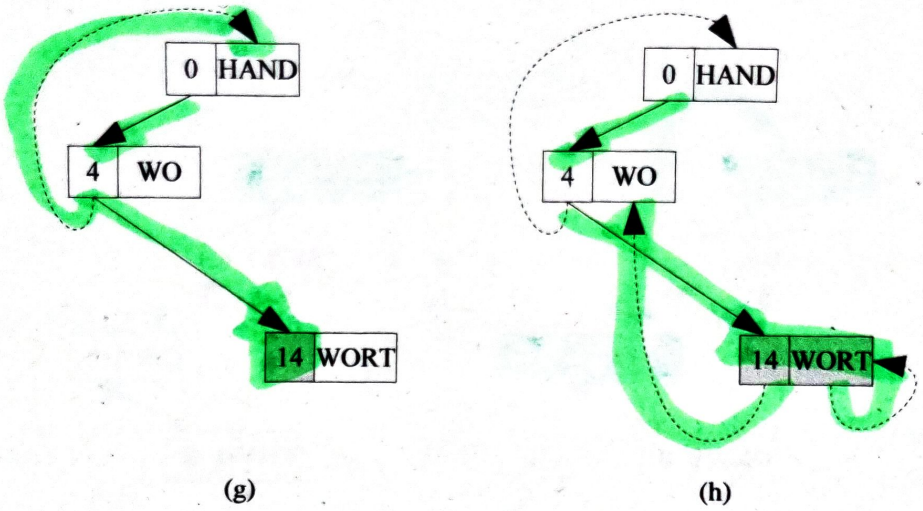
Abbildung 105: Einfügen in Patricia-Baum (cont.)



(e) (f)

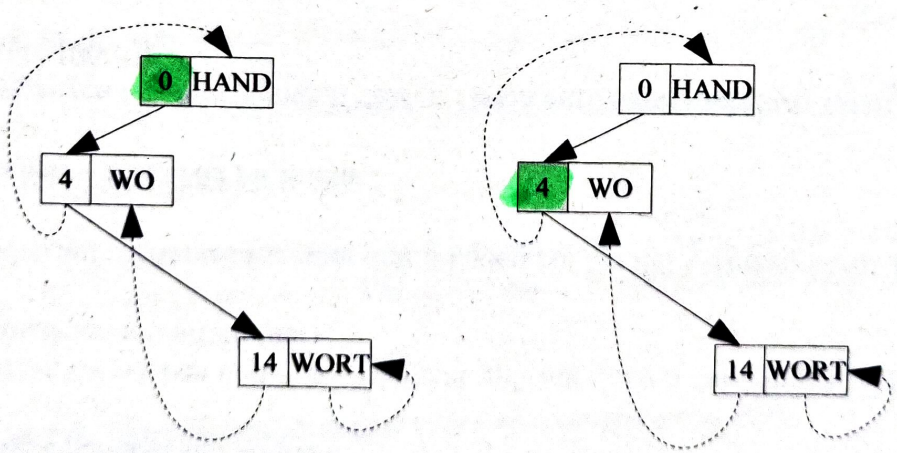
Abbildung 105: Einfügen in Patricia-Baum (cont.)

© R.-R. Grigat 2003-2019



(g) (h)

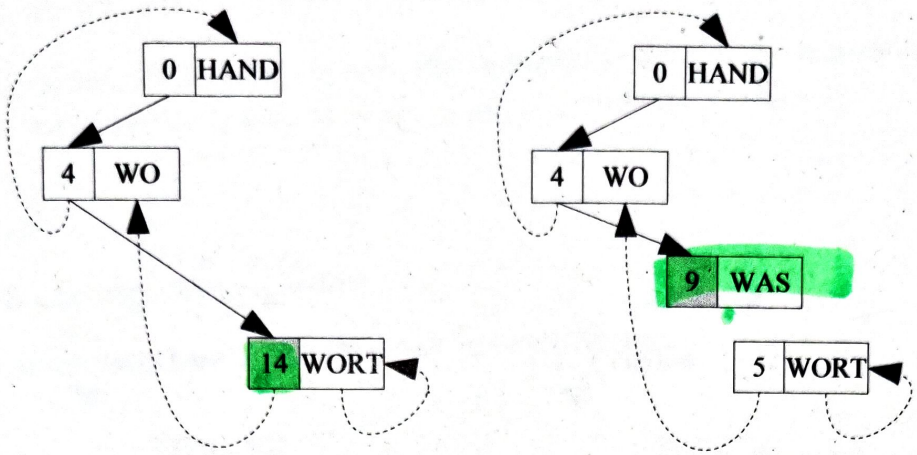
Abbildung 105: Einfügen in Patricia-Baum (cont.)



(i) (j)

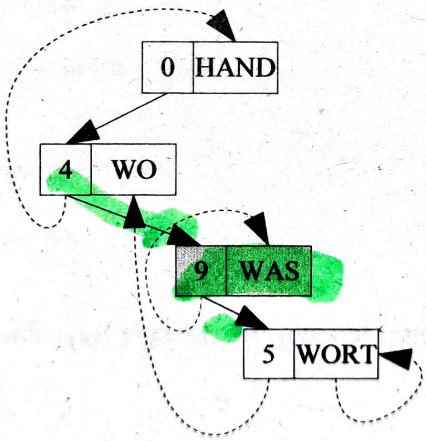
Abbildung 105: Einfügen in Patricia-Baum (cont.)

© R.-R. Grigat 2003-2019



(k) (l)

Abbildung 105: Einfügen in Patricia-Baum (cont.)



(m)

Abbildung 105: Einfügen in Patricia-Baum (cont.)

0.12.5.3 Löschen von Elementen aus Patricia-Bäumen

Beispiel: Entfernen von WO HAND

- Suchen von WO im Baum
- Vorgängerelement (Element, dessen Faden auf WO zeigt (5, WORT)) merken (Abb. 106(a))

© R.-R. Grigat 2003-2019

- WO durch Eintrag aus Vorgängerelement (WORT) ersetzen (Abb. 106(b))
- Vorgängerelement entfernen
- Zeiger auf soeben entferntes Element ggf. auf dessen Nachfolger, Positionswert aktualisieren (entfällt hier)
- Faden auf entferntes Element durch Faden auf WORT ersetzen (Abb. 106(c))
- Suchen von HAND im Baum
- Vorgängerelement (Element, dessen Faden auf HAND zeigt (4, WORT)) merken (Abb. 106(d))
- HAND durch Eintrag aus Vorgängerelement (WORT) ersetzen (Abb. 106(e))
- Vorgängerelement entfernen
- Positionswert des Nachfolgers aktualisieren (Abb. 106(f))
- Zeiger auf soeben entferntes Element durch Zeiger auf WAS ersetzen
- Faden auf entferntes Element durch Faden auf WORT ersetzen (Abb. 106(g))

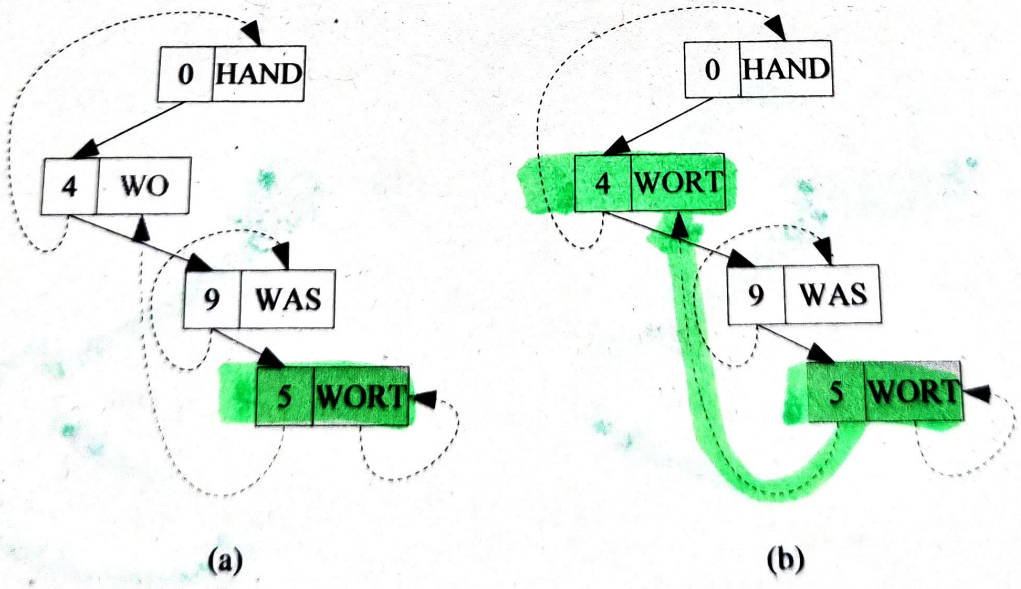
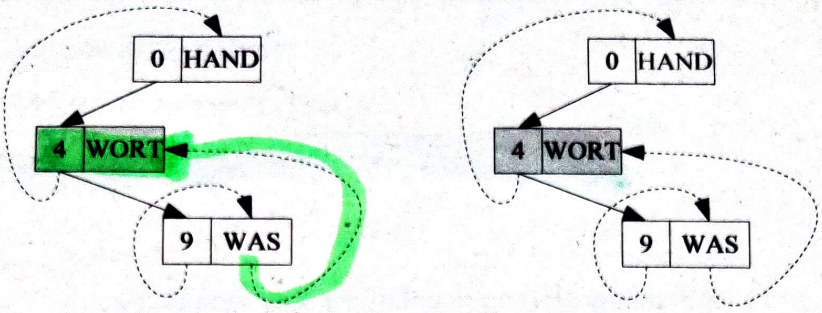


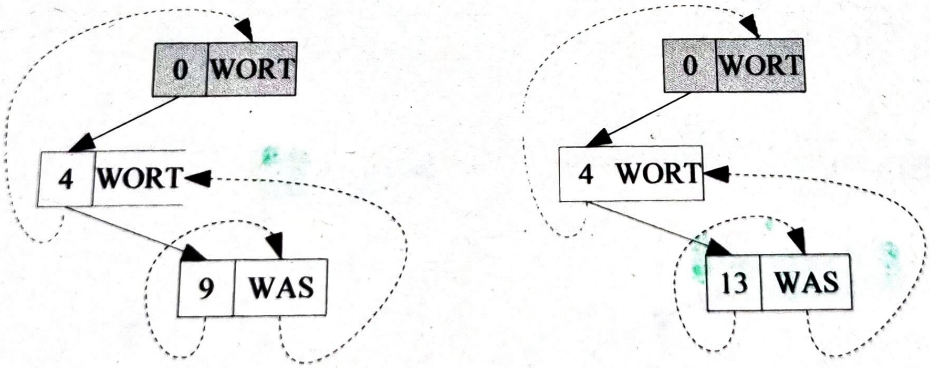
Abbildung 106: Löschen in Patricia-Baum

© R.-R. Grigat 2003-2019



(c) (d)

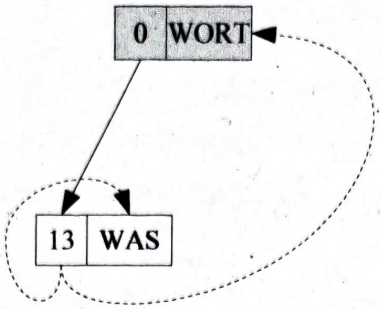
Abbildung 106: Löschen in Patricia-Baum (cont.)



(e) (f)

Abbildung 106: Löschen in Patricia-Baum (cont.)

© R.-R. Grigat 2003-2019



(g)

Abbildung 106: Löschen in Patricia-Baum (cont.)

0.12.5.4 Komplexitätsanalyse

Zeitkomplexität von Patricia-Bäumen

- Anzahl an Knoten im Baum: n
- Länge des Knoteninhalts: l_k
- Suche, d.h. Anzahl Bitvergleiche: $T_{insert_{\text{worst}}}, T_{insert_{\text{avg}}}, T_{find_{\text{worst}}}, T_{find_{\text{avg}}}(n) \in O(\log n)$
- Vergleich des Knoteninhalts: $T_{insert_{\text{worst}}}, T_{insert_{\text{avg}}}, T_{find_{\text{worst}}}, T_{find_{\text{avg}}}(l_k) \in O(l_k)$
- Gesamt: $O(\log n) + O(l_k)$
  - Vorteil gegenüber Tries: Logarithmische Komplexität (bezogen auf Anzahl der gespeicherten Elemente) auch im schlechtesten Fall

0.12.5.5 Anwendungsbeispiel von Patricia-Bäumen, Tries

Problemstellung

- Gegeben: *Quellfolge* ist (meist sehr lange) Liste $Q = \langle a_0, \ldots, a_{q-1} \rangle$ aus $q \in \mathbb{N}$ Zeichen a_i mit Index $0 \leq i \leq q-1$

© R.-R. Grigat 2003-2019

- Gegeben: *Suchfolge* ist (meist kurze) Liste $S = \langle b_0, \ldots, b_{s-1} \rangle$ aus $s \leq q$ Zeichen b_j mit Index $0 \leq j \leq s-1$, typisch $s \ll q$
- Gesucht: *Alle* Indexwerte bzw. Positionen i mit $0 \leq i \leq q-s$, an denen Suchfolge S in Quellfolge Q auftritt $< a_i, \ldots, a_{i+s-1} > = < b_0, \ldots, b_{s-1} >$

Beispiel

- Suche nach Wörten in Text
- Suche nach bestimmten Mustern in DNA-Sequenz

0.12.5.6 Naiver String-Matcher (Brute Force)

Naiver Algorithmus

- Prüfe für alle Indexwerte bzw. Positionen i in Quellfolge Q mit $0 \leq i \leq q-s$, ob dort Suchfolge S vorliegt $< a_i, \ldots, a_{i+s-1} > = < b_0, \ldots, b_{s-1} >$
- Zeitkomplexität der Suche: $O(q)$

Beschleunigung der Suche:

- Suchindex verwenden

0.12.5.7 Suchindex

Idee:

- *ASCII-Bitrepräsentationen* aller q Teilfolgen $< a_i, \ldots, a_{q-1} >$ der Quellfolge Q für $0 \leq i < q$ als *Schlüssel* der Länge $q-i$ verwenden
- Alle q Schlüssel zum Aufbau von Trie oder Patricia-Baum verwenden
- Die Schlüssel sind mit $O(q)$ meist sehr lang. Abhilfe im Patricia-Baum:
  - Indexwert i anstelle der Schlüssel in den Knoten speichern
  - Zusätzlich nur einmal gesamte ASCII-Bitrepräsentation der Quellfolge $< a_0, \ldots, a_{q-1} >$ speichern

© R.-R. Grigat 2003-2019

0.12.5.8 Indexsuche in Patricia-Baum

Suche nach Suchfolge S in Quellfolge Q

- Patricia-Baum: (Suchfolge S enthalte *kein* Terminierungssymbol)
  1. Suche der Suchfolge S als Präfix der Schlüssel zu den gespeicherten Indizes in Zeit $O(\log q)$
  2. Für ersten gefundenen Schlüsselkandidaten Vergleich der Suchfolge S mit Präfix der Länge s in $O(s)$
  3. Stimmen Präfix und Suchfolge S überein, so zeigen *alle* Fäden des zugehörigen Teilbaums unterhalb des letzten in Schritt 1 durchgeführten Bitvergleichs auf Indizes i für Suchfolgen S in Quellfolge Q. Ansonsten ist S in Q nicht enthalten.

Ergebnis:

- Zeitkomplexität pro Treffer: $O(s+\log q)$
- Zeitkomplexität für $O(q)$ Treffer: $O(s+q\log q)$

© R.-R. Grigat 2003-2019

0.13 Sortieralgorithmen

0.13.1 Einleitung

0.13.1.1 Sortierproblem

Definition: Sortierproblem

- Gegeben: $< e_1, \ldots, e_n >$
- Gesucht: Permutation $< e_{i_1}, \ldots, e_{i_n} >$, so dass $\text{key}(e_{i_1}) \leq \text{key}(e_{i_2}) \leq \ldots \leq \text{key}(e_{i_n})$

0.13.1.2 Einteilung von Sortieralgorithmen

Kriterien zur Einteilung von Sortieralgorithmen:

- Interne Verfahren
  - Alle Daten befinden sich gleichzeitig im Hauptspeicher
- Externe Verfahren
  - Nur Teilmenge der Datensätze wird jeweils geladen
- Methodisch
  - Einfache Verfahren
    - Sortieren durch Einfügen
    - Sortieren durch Auswählen
    - Bubblesort
  - Divide-And-Conquer (Teile-und-Herrsche)
    - Quicksort
    - Mergesort
  - Fachverteilen
    - Bucketsort
    - Radixsort
- Effizienz
  - Langsam: $O(n^2)$

© R.-R. Grigat 2003-2019

  - Schnell: $O(n \log n)$

- Speicherbedarf
  - Ein gemeinsames Array für Ausgangs- und Zielfolge
  - Getrennte Arrays für Ausgangs- und Zielfolge
  - Nicht-lineare Datenstruktur, die zusätzlichen Speicher benötigt (für Zeiger)
- Stabile Verfahren
  - Reihenfolge von Datensätzen mit gleichem Schlüsselwert wird nicht verändert

0.13.1.3 Komplexität von Sortieralgorithmen

Kostenmaß

- Grundlegende Operationen beim Sortieren:
  1. Vergleich zweier Schlüssel
  2. Vertauschen zweier Datensätze
- Diese Operationen werden zur Bestimmung der Komplexität gezählt

© R.-R. Grigat 2003-2019

0.13.2 Einfache Sortierverfahren

0.13.2.1 Beispiel zu Sortieren durch Auswählen

Beispiel: Sortieren der Folge $< 2,8,1,6,7,3,4,5 >$ *durch Auswählen*

Vertausche 1. Element (i=0) mit Minimum von 2,8,1,6,7,3,4,5
Vertausche 2. Element mit Minimum von 8,2,6,7,3,4,5
Vertausche 3. Element mit Minimum von 8,6,7,3,4,5
Vertausche 4. Element mit Minimum von 6,7,8,4,5
Vertausche 5. Element mit Minimum von 7,8,6,5
Vertausche 6. Element mit Minimum von 8,6,7
Vertausche 7. Element mit Minimum von 8,7

2	8	*1*	6	7	3	4	5
1	*8*	*2*	6	7	3	4	5
1	**2**	*8*	6	7	*3*	4	5
1	**2**	**3**	*6*	7	8	*4*	5
1	**2**	**3**	**4**	*7*	8	6	*5*
1	**2**	**3**	**4**	**5**	*8*	*6*	7
1	**2**	**3**	**4**	**5**	**6**	*8*	*7*
1	**2**	**3**	**4**	**5**	**6**	**7**	8

Tabelle 26: Beispiel zu Sortieren durch Auswählen

0.13.2.2 Sortieren durch Auswählen (Selectionsort)

Sortieren des Arrays a der Länge *N*

```
1  template <typename T>
2  void selectionSort(T *a)  //Minimum in unsortierten Zahlen auswählen
3  {
4    for (int i = 0; i<(N-1); i++)
5    {
6      // Invariante: a[0..i-1] fertig sortiert  (für i>0)
7      int min_pos = i;
8      // suche nun Minimum in a[i..N] (unsortierter Teilliste)
9      for (int j = i; j<N; j++)
10     {
11       // Invariante: a[min_pos] minimal in a[i..j-1] (für j>i)
12       if (a[min_pos] > a[j])
```

© R.-R. Grigat 2003-2019

```
13        {
14          min_pos = j;
15        }
16      }
17      // Tausche Minimum der Teilliste a[i..N] mit Position a[i]
18      swap(a[i], a[min_pos]);
19    }
20  }
```

0.13.2.3 Komplexität des Sortierens durch Auswählen (Selectionsort)

Komplexität von Selectionsort

- $C(n) \in O(n^2)$ (Comparisons)
- $X(n) \in O(n)$ (Exchanges)
- $T(n) = C(n) + X(n) \in O(n^2) + O(n) = O(n^2)$

© R.-R. Grigat 2003-2019

0.13.2.4 Beispiel zu Sortieren durch Einfügen

Beispiel: Sortieren der Folge $< 7,1,3,6,4,2,8,5 >$ *durch Einfügen*

1. Füge Element (1) in Liste mit dem 1. Element $< 7 >$ ein.
2. Füge Element (3) in sortierte Liste $< 1,7 >$ ein
3. Füge Element (6) in sortierte Liste $< 1,3,7 >$ ein
4. Füge Element (4) in sortierte Liste $< 1,3,6,7 >$ ein
5. Füge Element (2) in sortierte Liste $< 1,3,4,6,7 >$ ein
6. Füge Element (8) in sortierte Liste $< 1,2,3,4,6,7 >$ ein
7. Füge Element (5) in sortierte Liste $< 1,2,3,4,6,7,8 >$ ein

7	***1***	3	6	4	2	8	5
1	**7**	***3***	6	4	2	8	5
1	***3***	**7**	***6***	4	2	8	5
1	**3**	***6***	**7**	***4***	2	8	5
1	**3**	***4***	**6**	**7**	***2***	8	5
1	***2***	**3**	**4**	**6**	**7**	***8***	5
1	**2**	**3**	**4**	**6**	**7**	***8***	***5***
1	**2**	**3**	**4**	***5***	**6**	**7**	**8**

Tabelle 27: Beispiel zu Sortieren durch Einfügen

© R.-R. Grigat 2003-2019

0.13.2.5 Sortieren durch Einfügen (Insertion Sort)

Sortieren des Arrays a der Länge N

```
1  template <typename T>
2  void insertionSort(T *a) //Zahlen in sortierte Folge einfügen
3  {
4    for (int j = 1; j<N; j++)
5    {
6     // Invariante: a[0..j-1] fertig sortiert
7     T key = a[j];
8     // fuege a[j] in sortierte Sequenz a[0..j-1] ein
9     int i = j-1;
10    while ((i>=0) && (a[i]>key))
11    {
12      // Invariante für i<j-1:  a[i+2..j]>key und sortiert,
13      //                        a[i+1]=a[i+2], a[i]>key
14      a[i+1] = a[i];
15      i--;
16    }
17    // Invariante: i==-1 oder a[i]<=key
18    a[i+1] = key;
19   }
20 }
```

0.13.2.6 Komplexität des Sortierens durch Einfügen (Insertionsort)

Komplexität von Insertionsort

- $C(n) \in O(n^2)$ (Comparisons)
- $M(n) \in O(n^2)$ (Move-Operationen)
- $T(n) = C(n) + M(n) \in O(n^2) + O(n^2) = O(n^2)$

© R.-R. Grigat 2003-2019

0.13.2.7 Beispiel zu Sortieren durch Austauschen (Bubblesort)

Beispiel: Sortieren der Folge $< 6,2,4,1,8,7,5,3 >$ *durch Austauschen*

6	2	4	1	8	7	*5*	*3*
6	2	4	1	8	*7*	*3*	5
6	2	4	1	*8*	*3*	7	5
6	2	4	*1*	*3*	8	7	5
6	2	*4*	*1*	3	8	7	5
6	*2*	*1*	4	3	8	7	5
6	*1*	2	4	3	8	7	5
1	6	2	4	3	8	7	5
1	6	2	4	3	8	*7*	*5*
1	6	2	4	3	*8*	*5*	7
1	6	2	4	*3*	*5*	8	7
1	6	2	*4*	*3*	5	8	7
1	6	*2*	*3*	4	5	8	7
1	*6*	*2*	3	4	5	8	7
1	**2**	6	3	4	5	8	7
1	**2**	6	3	4	5	*8*	*7*
1	**2**	6	3	4	*5*	*7*	8
1	**2**	6	3	*4*	*5*	7	8
1	**2**	6	*3*	*4*	5	7	8
1	**2**	*6*	*3*	4	5	7	8
1	**2**	**3**	6	4	5	7	8
1	**2**	**3**	6	4	5	*7*	*8*
1	**2**	**3**	6	4	*5*	*7*	8
1	**2**	**3**	6	*4*	*5*	7	8
1	**2**	**3**	*6*	*4*	5	7	8
1	**2**	**3**	**4**	6	5	7	8
1	**2**	**3**	**4**	6	5	*7*	*8*
1	**2**	**3**	**4**	6	*5*	*7*	8
1	**2**	**3**	**4**	*6*	*5*	7	8
1	**2**	**3**	**4**	**5**	6	7	8
1	**2**	**3**	**4**	**5**	6	*7*	*8*
1	**2**	**3**	**4**	**5**	*6*	*7*	8
1	**2**	**3**	**4**	**5**	**6**	7	8
1	**2**	**3**	**4**	**5**	**6**	*7*	*8*
1	**2**	**3**	**4**	**5**	**6**	**7**	**8**

Tabelle 28: Beispiel zu Sortieren durch Austauschen

© R.-R. Grigat 2003-2019

0.13.2.8 Sortieren durch Austauschen (Bubblesort)

Sortieren des Arrays a der Länge N

```
template<typename T>
void bubbleSort(T *a){
  for (i=0; i<N-1, i++)
  {
  // Invariante für i>0:
  // a[0..i-1] fertig sortiert und a[i-1]<=a[i..N-1]
    for (j=N-1; j>i; j--)
    {
    // Invariante: a[j] ist Minimimum von a[j..N-1]
      if (a[j] < a[j-1]) swap(a[j], a[j-1]);
    }
  }
}
```

0.13.2.9 Komplexität des Sortierens durch Austauschen (Bubblesort)

Komplexität von Bubblesort

- $C(n) \in O(n^2)$ (Comparisons)
- $X(n) \in O(n^2)$ (Exchange-Operationen)
- $T(n) = C(n) + X(n) \in O(n^2) + O(n^2) = O(n^2)$

© R.-R. Grigat 2003-2019

0.13.3 Sortieren mittels „Teile-und-Herrsche“

0.13.3.1 „Teile-und-Herrsche“ (Divide-and-Conquer)

Teile-und-Herrsche

```
if (Objektmenge klein genug)
{
  löse Problem direkt;
}
else
{
  Divide();
  Conquer();
  Merge();
}
```

```
Divide()
{
  Zerlege Menge in mehrere Teilmengen
    (wenn möglich, gleicher Größe);
}
```

```
Conquer()
{
  Löse Problem rekursiv
    für jede Teilmenge;
}
```

```
Merge()
{
  Berechne aus Lösungen
    für die Teilmengen
    die Gesamtlösung;
}
```

© R.-R. Grigat 2003-2019

0.13.3.2 Mergesort

Sortieren des Arrays a der Länge N

- Rekursive Definition per Divide-and-Conquer
- Zunächst Aufruf von `mergeSort(a, 0, N)`

```
1  template <typename T>
2  void mergeSort(T *a, int l, int r)   // sortiere a[l..r-1]
3  {
4    if (l<(r-1))
5    {
6  //==> Divide: Aufteilung in Teilprobleme
7      int m = (l+r)/2; // int-Division rundet abwaerts
8  //==> Conquer: Rekursiv Problemlösung starten
9      mergeSort(a, l, m); // sortiere a[l..m-1]
10     mergeSort(a, m, r); // sortiere a[m..r-1]
11 //==> Merge: Teillösungen zusammenfügen
12     merge(a, l, m, r);  // Zusammenfuegen der Teillisten
13                         // a[l..m-1] und a[m..r-1]
14   }
15 }
```

0.13.3.3 Beispiel zu Mergesort

Beispiel: Sortieren der Folge $< 2,8,1,6,7,3,4,5 >$ *mit Mergesort*

Divide (Zerlegung), Conquer (Aufruf nächster Stufe):

Divide 1, Conquer 1	2	8	1	6	7	3	4	5
Divide 2, Conquer 2	2	8	1	6	7	3	4	5
Divide 3, Conquer 3	2	8	1	6	7	3	4	5
	2	8	1	6	7	3	4	5

Merge (Zusammensortieren der Teillisten):

Merge 3	2	8	1	6	3	7	4	5
Merge 2	1	2	6	8	3	4	5	7
Merge 1	1	2	3	4	5	6	7	8

Tabelle 29: Beispiel zu Mergesort

© R.-R. Grigat 2003-2019

0.13.3.4 Die Merge-Operation

Zusammenfügen der Teillisten a[l..m-1] und a[m..r-1]

```
template <typename T>
void merge (T *a, int l,
            int m, int r)
{
int n1 = m-l; //# links
int n2 = r-m; //# rechts
std::vector<T> llist;
std::vector<T> rlist;
for (int i=0; i<n1; i++)
{ // Kopie linker
  // Teilliste anlegen
 llist.push_back(a[l+i]);
}
for (int j=0; j<n2; j++)
{ // Kopie rechter
  // Teilliste anlegen
 rlist.push_back(a[m+j]);
}
```

```
int i=0; int j=0; int k=l;
while (k<r)
{
  if ((i<n1) && j<n2)
  // noch nicht am Ende
  // einer Teilliste
  {
    if (llist[i]<rlist[j])
    // kleineres Element
    // uebernehmen
    { a[k] = llist[i];
      i++; k++;
    }
    else
    { a[k] = rlist[j];
      j++; k++;
  } }
```

```
  else
// am Ende einer
```

© R.-R. Grigat 2003-2019

```
40 // Teilliste die
41 // restlichen Elemente
42 // der anderen Teilliste
43 // uebernehmen
44   {
45     while (i < n1)
46     { a[k] = llist[i];
47       i++;
48       k++;
49     }
50     while (j < n2)
51     { a[k] = rlist[j];
52       j++;
53       k++;
54 }}} }
```

0.13.3.5 Komplexität von Mergesort

Komplexität bei Implementierung mit Liste

$$T(n) = T_{\text{Divide}} + T_{\text{Conquer}} + T_{\text{Merge}}$$
$$\in \begin{cases} O(1) & \text{falls } n = 1 \\ O(n) + 2 \cdot T(\frac{n}{2}) + O(n) & \text{falls } n > 1 \end{cases}$$

Komplexität bei Implementierung mit Array

$$T(n) = T_{\text{Divide}} + T_{\text{Conquer}} + T_{\text{Merge}}$$
$$\in \begin{cases} O(1) & \text{falls } n = 1 \\ O(1) + 2 \cdot T(\frac{n}{2}) + O(n) & \text{falls } n > 1 \end{cases}$$

Gesamtkomplexität (Auflösen der Rekursionsgleichung)

- Länge der Teillisten in jedem Rekursionsschritt halbiert
- n Elemente $\Rightarrow \log n$ Rekursionsschritte aus $O(n)$
- $\Rightarrow T(n) \in \log n \cdot O(n) = O(n \log n)$

© R.-R. Grigat 2003-2019

0.13.3.6 Beispiel zu Quicksort

Beispiel: Sortieren der Folge $< 2,8,1,6,7,3,4,5 >$ *mit Quicksort*

2	*8*	1	6	7	3	*4*	*5*
l							r-1
i							j
	i						j
	i					j	
2	4	1	*6*	7	*3*	8	*5*
	i					j	
		i				j	
			i			j	
			i		j		
2	4	1	3	*7*	6	8	*5*
			i		j		
				i	j		
2	*4*	*1*	*3*	**5**	6	8	7
l				m			r-1
l			r-1		1		r-1
i			j				
	i		j				
	i	j					
2	1	*4*	*3*	**5**	6	8	*7*
		i					
		j					
2	*1*	**3**	**4**	**5**	6	8	*7*
l		m	r-1				
l	r-1						
i	j						
i							
j							
1	**2**	**3**	**4**	**5**	6	8	*7*

Tabelle 30: Beispiel zu Quicksort für linke Teilliste. Rechte Teilliste entsprechend.

© R.-R. Grigat 2003-2019

0.13.3.7 Quicksort

Sortieren des Arrays a der Länge N

- Rekursive Definition per Divide-and-Conquer
- Pro Schritt (willkürliche) Wahl eines Pivotelements x und Erstellung von zwei nur bzgl. x sortierten Teilfolgen mit $a[l \ldots m-1] \leq x = a[m] \leq a[m+1 \ldots r-1]$
- Zunächst Aufruf von `quickSort(a, 0, N)`

```
template <typename T>
void quickSort(T *a, l, r)   // sortiere a[l..r-1]
{
  if (l<r-1)
  {
    int m = partition(a, l, r);     // Divide
       // Es gilt nun a[l..m-1]<=a[m]<=a[m+1..r-1]
       // mit zwei noch in sich unsortierten Teilfolgen
       // a[l..m-1] und a[m+1..r-1]
    quickSort(a, l, m);             // Conquer: sortiere a[l..m-1]
    quickSort(a, m+1, r);           // Conquer: sortiere a[m+1..r-1]
} }                                 // Merge nicht erforderlich
```

0.13.3.8 Divide: Die Partition-Operation

Zerlegen des Feldes a

```
template <typename T>
int partition(T *a, int l, int r)
{ // Als Pivotelement x wählen wir das
  // Array-Element mit größtem Index
  T& x = a[r-1]; // Pivotelement
  int i = l;     // Linksindex
  int j = r-1;   // Rechtsindex
  while (i<j) // äußere Schleife
  {//Ab 2. Schleifendurchlauf gilt:
   //a[l..i]<=x<=a[j..r-1], l<i<j<=r-1

    while ((i<j) && (a[i] <= x))
    {
      i++; // Durchlauf von links
```

© R.-R. Grigat 2003-2019

```
15      }
16     //x<a[i] oder
17     //(i==j und x<=a[i]=a[j] und 1<i)

19      while ((i<j) && (x <= a[j]))
20      {
21        j--;  // Durchlauf von rechts
22      }
23    // a[j] < x < a[i] oder
24    // (i==j und (x<=a[i]=a[j] und 1<i
25    //       oder x< a[i]=a[j] ))
26      if (i<j) swap (a[i], a[j]);
27    } // Ende äußere Schleife,
28    // i==j, a[1..i-1]<=x<=a[i..r-1]
29    swap(a[i], a[r-1]);
30    // Pivotelement mit a[i] tauschen
31    // Invariante:
32    // a[1..i-1]<=a[i]<=a[i+1..r-1]
33    // mit zwei unsortierten Teilfolgen
34    return i;
35  } // Ende von partition
```

0.13.3.9 Komplexität von Quicksort

Komplexität von Quicksort

- Falls Teillisten jeweils gleich groß: $T_{\text{avg}}(n) \in O(n \log n)$
- Falls eine Teilliste jeweils nur aus einem Element besteht oder leer ist: $T_{\text{worst}}(n) \in O(n^2)$
  - Problem insbesondere bei sortierten bzw. invers sortierten Listen

Strategien zur Vermeidung unbalancierter Teillisten

- Element an mittlerer Position als Pivotelement
  - sinnvoll, falls Liste annähernd sortiert
- Element an zufälliger Position als Pivotelement
  - keine Annahmen über Sortierungsgrad der Liste notwendig

© R.-R. Grigat 2003-2019

- Element mit mittlerem Wert von drei zufälligen Positionen (Median) als Pivotelement
  - schließt aus, dass zufällig kleinstes oder größtes Element gewählt wird

0.13.4 Sortieren mit Bäumen partieller Ordnung (Heaps)

0.13.4.1 Heapsort

Idee

1. Einfügen von n Elementen in Heap: $T(n) \in O(n \log n)$
2. n-mal Entnahme des Minimums aus Heap: $T(n) \in O(n \log n)$

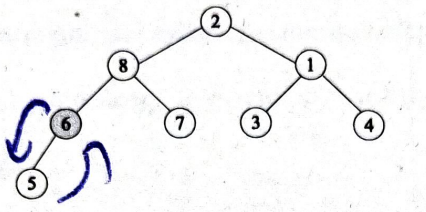
Verfeinerung der Idee zu Heapsort

- Konstruiere Heap im Array der unsortierten Eingabedaten.
- Fasse dazu Daten im Array als links-vollständigen Binärbaum auf und stelle partielle Ordnung her.
  - Jedes Blatt besitzt bereits für sich partielle Ordnung
  - Wende auf alle inneren Knoten (beginnend beim letzten inneren Knoten) Sift an, Resultat ist Heap
- Erstelle aus Heap eine sortierte Liste
  - Tausch des Wurzelelements mit letztem Knoten des Heaps
  - Letzter Knoten des Heaps gehört ab jetzt nur noch zur Liste
  - Sift auf Wurzel anwenden für die partielle Ordnung
  - Wiederhole letzte drei Schritte bis Heap leer ist
- Resultat ist sortierte Liste aller Elemente
- Falls aufsteigende Sortierung gewünscht: Maximum-Heap verwenden

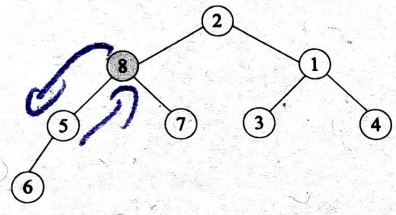
© R.-R. Grigat 2003-2019

0.13.4.2 Beispiel zu Heapsort:

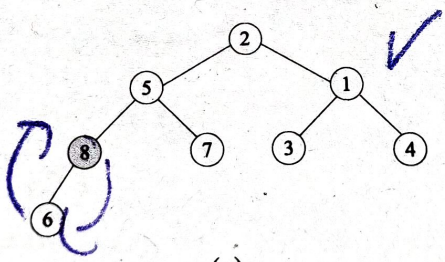
Beispiel: Sortieren der Folge $< 2,8,1,6,7,3,4,5 >$ *mit Heapsort*



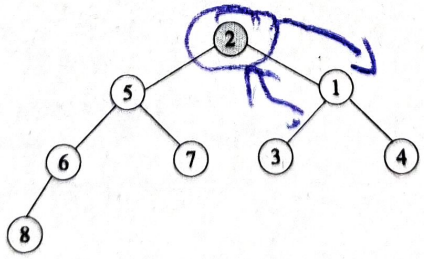
(a)



(b)



(c)



(d)

Abbildung 107: Herstellung partieller Ordnung: *Sift* auf alle *inneren* Knoten anwenden, beginnend beim letzten inneren Knoten

© R.-R. Grigat 2003-2019

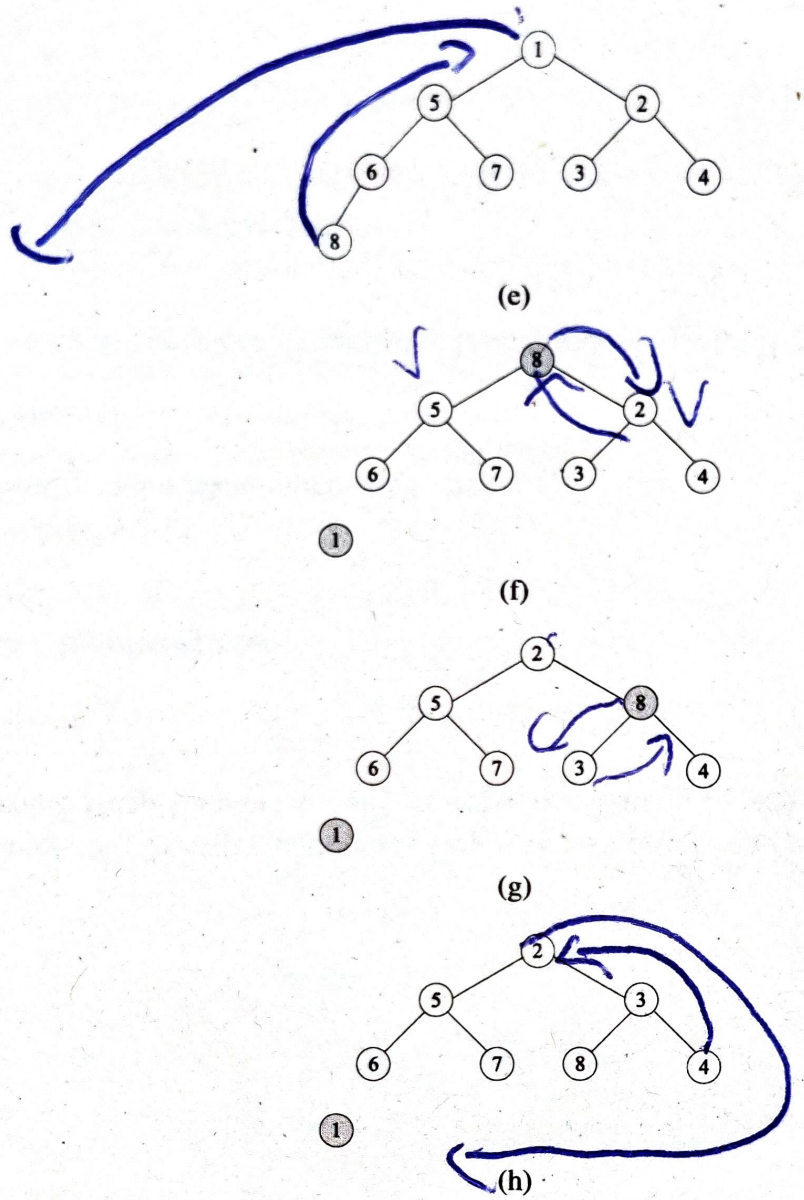
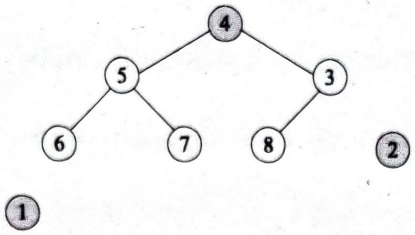
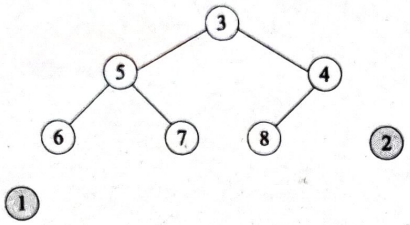


Abbildung 107: Transformation des Heaps in sortierte Liste durch Tausch von Wurzel und letztem Heap-Element sowie Sift der neuen Wurzel

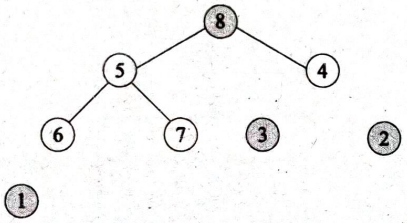
© R.-R. Grigat 2003-2019



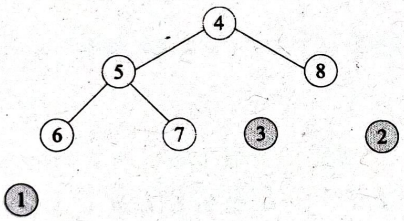
(i)



(j)



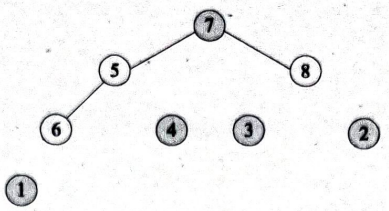
(k)



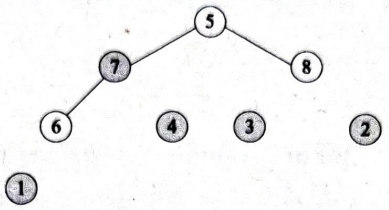
(l)

Abbildung 107: Transformation des Heaps in sortierte Liste durch Tausch von Wurzel und letztem Heap-Element sowie Sift der neuen Wurzel (cont.)

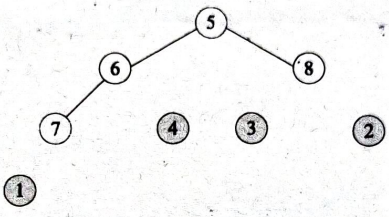
© R.-R. Grigat 2003-2019



(m)



(n)



(o)

Abbildung 107: Transformation des Heaps in sortierte Liste durch Tausch von Wurzel und letztem Heap-Element sowie Sift der neuen Wurzel (cont.)

0.13.4.3 Baumsortieren

Idee:

- Einfügen der Elemente in AVL-Baum T
- *inorder(T)* liefert sortierte Folge
- Nachteil: komplexe dynamische Datenstruktur erforderlich

© R.-R. Grigat 2003-2019

0.13.5 Sortieren durch Fachverteilen

0.13.5.1 Bucketsort

Idee

- Annahme: $key(a_n) \leq N-1 \; \forall \; n$ also Zahl der Schlüssel endlich und bekannt
- Hashtabelle (offenes Hashing) der Größe N für Elemente verwenden
- Teillisten sortieren mit Sortieren durch Einfügen

0.13.5.2 Beispiel zu Bucketsort

Beispiel: Sortieren der Folge $<1,2,6,7,3,5,3,7>$ *mit Bucketsort*

- $N=8$

0	$\diamond$
1	$<1>$
2	$<2>$
3	$<3,3>$
4	$\diamond$
5	$<5>$
6	$<6>$
7	$<7,7>$

Tabelle 31: Beispiel zu Bucketsort: Hashtabelle der Größe 8

0.13.5.3 Radixsort (Distributionsort)

Idee

- Ziffernfolgen zur Basis m ziffernweise mit Bucketsort sortieren
- Beginn mit Sortierung nach der niederwertigsten Ziffer
- Konkatenation der nach niederwertigsten Ziffer sortierten Teillisten
- Sortierung nach zweitniederwertigster Ziffer usw.

© R.-R. Grigat 2003-2019

- Größe der einzelnen Hashtabellen: $N = m$
- *Lineare* Laufzeit $O(n)$ (!), aber nur für konstante Stellenzahl der Schlüssel, also für begrenzte Schlüsselmenge

0.13.5.4 Beispiel zu Radixsort (Distributionsort)

Radixsort der Folge $< 21,36,17,23,35,13,37 >$

- Ergebnis Schritt 1: $< 21,23,13,35,36,17,37 >$
- Ergebnis Schritt 2: $< 13,17,21,23,35,36,37 >$

	(a)		(b)
0	$\diamond$	0	$\diamond$
1	$< 21 >$	1	$< 13,17 >$
2	$\diamond$	2	$< 21,23 >$
3	$< 23,13 >$	3	$< 35,36,37 >$
4	$\diamond$	4	$\diamond$
5	$< 35 >$	5	$\diamond$
6	$< 36 >$	6	$\diamond$
7	$< 17,37 >$	7	$\diamond$
8	$\diamond$	8	$\diamond$
9	$\diamond$	9	$\diamond$

Tabelle 32: Beispiel zu Radixsort (Distributionsort)

© R.-R. Grigat 2003-2019

0.13.5.5 Vergleich von Sortierverfahren

	unsortiert		vorsortiert	
$n =$	$5 \cdot 10^6$	$20 \cdot 10^6$	$5 \cdot 10^6$	$20 \cdot 10^6$
QuickSort	0,94	4,05	0,26	1,13
MergeSort	1,77	7,66	1,13	4,94
HeapSort	4,19	22,41	1,55	6,67
DistributionSort	0,53	2,10	0,49	2,03
BubbleSort			0,01	0,06
InsertionSort			0,01	0,06

Tabelle 33: Laufzeitvergleich schneller Sortierverfahren in Sekunden für $5 \cdot 10^6$ und $20 \cdot 10^6$ zufällig gewählte Datensätze nach [?]

0.13.5.6 Vergleich von Sortierverfahren (2)

- Bei geringen Datenmengen einfachen Algorithmus verwenden (InsertionSort, SelectionSort, BubbleSort)
- Zum Einsortieren in nahezu sortierten Datenbestand Insertionsort oder Bubblesort verwenden
- Für sehr viele zufällig verteilte und oft zu sortierende Daten Distributionsort anpassen
- In Standardsituationen sind Mergesort und Heapsort zu empfehlen
- Quicksort ist schnell, birgt aber das Risiko ungünstiger Datenverteilung
- YouTube - Sortieren vertont `https://www.youtube.com/watch?v=kPRAOW1kECg`

© R.-R. Grigat 2003-2019